

Campus Service Request Management System

Complete Pass, Credit and Distinction Implementation

Taisen Marainump | Student ID 250692
Faculty of Business and Informatics
Department of Information Systems

Purpose and scope

The Node.js console application records campus ICT, maintenance, cleaning and general service requests, assigns responsibility, tracks work and preserves approved actions. One application extends its Pass foundation through Credit and the detailed Distinction requirements. No database is used. This report describes implemented and tested behaviour, rather than guaranteeing an assessment grade.

Component	Implemented features
Pass — 25 marks	Core classes, private fields, constructors, validation, requester relationships, arrays and integrated console workflow.
Credit — 10 marks	User/request inheritance, super(), specialised validation and overrides, role-controlled lifecycle, history and queries.
Distinction — 15 marks	Abstract request methods, polymorphism, four-file repositories, factory restoration, audit, reports and automated tests.

Class hierarchy and encapsulation

User is the base of StudentRequester, StaffRequester, ServiceOfficer, Technician and SystemAdministrator. ServiceRequest is an abstract-style domain base. Its three required methods throw when a subclass does not implement them. ICTSupportRequest, MaintenanceRequest and CleaningRequest implement specialised summaries, scores and target times. BasicServiceRequest preserves concrete Pass behaviour; GeneralServiceRequest covers the fourth category. Private fields protect common state and controlled methods validate changes.

Composition and controlled lifecycle

The manager stores users and requests in arrays. Requests refer to the original registered requester and assigned technician objects. The permitted flow is Submitted → Reviewed → Assigned → In Progress → Resolved → Closed; Cancelled is final. Requesters may edit/cancel only their own Submitted requests. Officers review, prioritise, assign and verify closure. Only the assigned technician starts work, records progress and resolves. Invalid changes leave state and history unchanged.

Polymorphism and abstraction

Menu option 25 loops through one request collection and calls `getRequestSummary()`, `calculatePriorityScore()` and `getTargetResolutionHours()` on each object. Concrete class implementations determine results. A report loop calls `generate()` on four report implementations. `ServiceRequest`, `RequestReport` and `StorageAdapter` define abstract-style contracts. Constructor chaining and separation of domain, manager, console, storage and reporting responsibilities support the assessment OOP objectives.

Four-file persistence and repositories

The data folder contains `users.json`, `serviceRequests.json`, `requestHistory.json` and `auditLog.json`, initially empty arrays. `FileRepositories.js` implements four repositories with `loadAll/saveAll/create/find/update` methods and request-specific queries. `FourFileStore` coordinates approved snapshots. `CampusServiceApp` invokes storage operations but never directly reads or writes JSON. Missing or empty files load empty arrays; inconsistent or malformed datasets produce clear errors without overwriting source data.

Object restoration and save recovery

`ServiceRequestFactory.createFromData` selects the saved `requestType` and reconstructs the correct subclass with specialised validation. Loading recreates User subclasses and shared references, restores request state/history and checks audit agreement. Saving validates records before writing. Temporary files and rename protect each file; a recovery journal preserves the prior four-file dataset. Failed or interrupted saves restore that dataset. The single-console design supports one running process, not concurrent writers.

Audit records

Each approved audit entry stores a unique `auditId`, actor ID/role, action, affected request ID, description, ISO UTC timestamp and outcome. Registration uses a null request ID and includes the affected user ID. Creation, details updates, priority changes, assignment, status/progress actions, cancellation, resolution and closure are recorded. Audit and request histories are persisted separately and validated together. Rejected actions do not create an Approved entry.

Management reports

Four concrete report implementations provide status/category/priority groups, urgent and overdue requests, technician assignments and completions, average resolution time and request volume by campus location. Array filter/map/reduce/sort operations implement the calculations. Completed means Resolved or Closed; elapsed resolution time ends at first resolution. No completions produces a null average. Overdue excludes resolved/closed/cancelled work. Target hours and subtype risk scores are illustrative academic assumptions, not official DWU commitments.

Verification evidence

All 53 automated tests passed on Node v24.19.0: 14 Pass, 12 Credit and 27 Distinction/console tests. Coverage includes construction, invalid values, duplicate identifiers, permissions, transitions, subtype behaviour, polymorphism, four-file saves, restored objects, missing/empty/corrupt files, real I/O errors, interrupted-save recovery, report calculations and actual console workflows. File tests use OS temporary directories and do not change normal application data. TEST_RESULTS.txt and examples contain actual execution evidence.

Demonstration and submission

Run npm test and npm run start:demo-data from the extracted project folder in the VS Code PowerShell terminal. Option 11 switches simulated users; option 10 exits. Follow TECHNICAL_DEFENCE.md to show the lifecycle, denied actions, reports, audit, polymorphism and restart/reload. GITHUB_AND_MOODLE.md explains copying to the existing repository, committing, pushing and submitting the final ZIP. Confirm the exact calendar deadline in Moodle.

Assumptions and limitations

The requirements source is the user-supplied DWU IS305 AT3 brief and detailed Pass, Credit and Distinction instructions. All data is simulated and example.test emails are synthetic. Role selection is a classroom simulation, not password authentication. No credentials, executable JavaScript or confidential records belong in JSON. Earlier single-file checkpoint data is not automatically imported. If a save fails, session changes remain in memory and require an administrator retry before exit.